

第7讲速查卡：从 RNN / LSTM 到 Attention / Transformer

阶段	核心机制	为什么出现	主要遗留问题
RNN	h_t 递归传递	需要让序列拥有上下文状态	长依赖与顺序计算
LSTM	门控 + Cell State	改善梯度与长期信息保留	仍然递归，时间维难完全并行
Seq2Seq	Encoder $\rightarrow$ 固定 Context $\rightarrow$ Decoder	输入序列到输出序列	固定向量信息瓶颈
Attention	相关性 $\rightarrow$ 权重 $\rightarrow$ 加权读取	让 Decoder 动态访问输入各位置	起初仍依附 Encoder-Decoder
Self-Attention	同一序列位置互相读取	直接建模序列内部关系	需要位置编码；长序列成本
Transformer	Self-Attention + FFN 主干	去掉循环主干，更利于并行扩展	标准注意力 n^2 成本

最小公式地图

RNN: $h_t = f(W_x x_t + W_h h_{(t-1)} + b)$
Attention 直觉: $\text{score} \rightarrow \text{softmax} \rightarrow \text{weights} \rightarrow \sum (\text{weight}_i \times \text{value}_i)$
Transformer: $\text{Embedding} + \text{Position} \rightarrow [\text{Self-Attention} + \text{FFN}] \times N \rightarrow \text{输出}$

概念	一句话
长期依赖	当前位置需要很久以前的信息
梯度消失	越往前反传，学习信号越接近 0
信息瓶颈	很多输入被迫压进一个固定长度表示
Attention	不只依赖“摘要”，而是直接按相关性读取多个位置
并行性	能否把大量 token/矩阵计算同时交给 GPU

常见误区与下一讲接口

误区	正确理解
LSTM=永久记忆	只是改善长期依赖，不保证永不忘
Attention=人类注意力	它是可微的数值加权机制
Transformer不需要顺序	需要位置编码等显式提供位置信息
Attention解决所有长上下文问题	n 成本和有效利用长上下文仍是难题
Transformer只是算法更强	与 GPU 并行和工程可扩展性高度相关

下一讲先学什么

先不急背 Q/K/V。先彻底理解三步：比较相关性 → 转成权重 → 按权重读取信息。然后再把 Query、Key、Value 分别接到这三步上。

记忆口令：RNN “传状态” → LSTM “管记忆” → Attention “直接查” → Transformer “让所有位置都这样查”